

What's in this course?

Module 1 – The Software Development Lifecycle

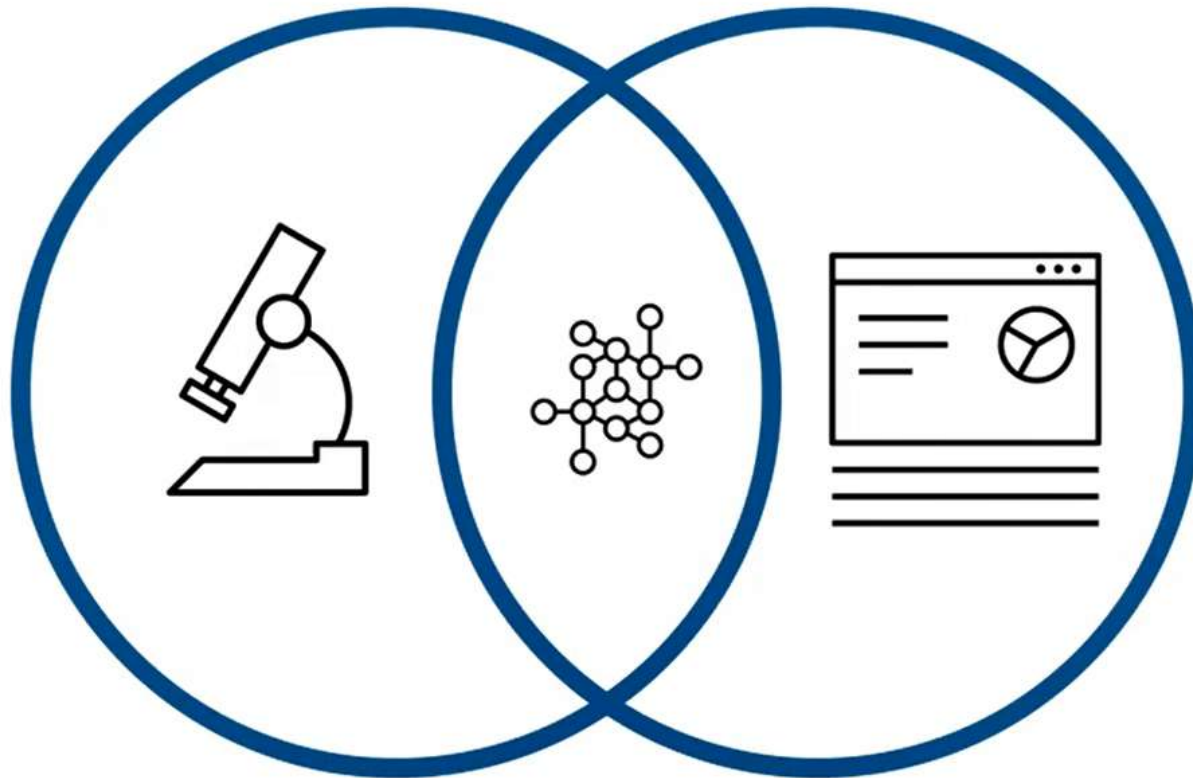
Module 2 – Introduction to Software Development

Module 3 – Basics of Programming

Module 4 – Software Architecture, Design, and Patterns

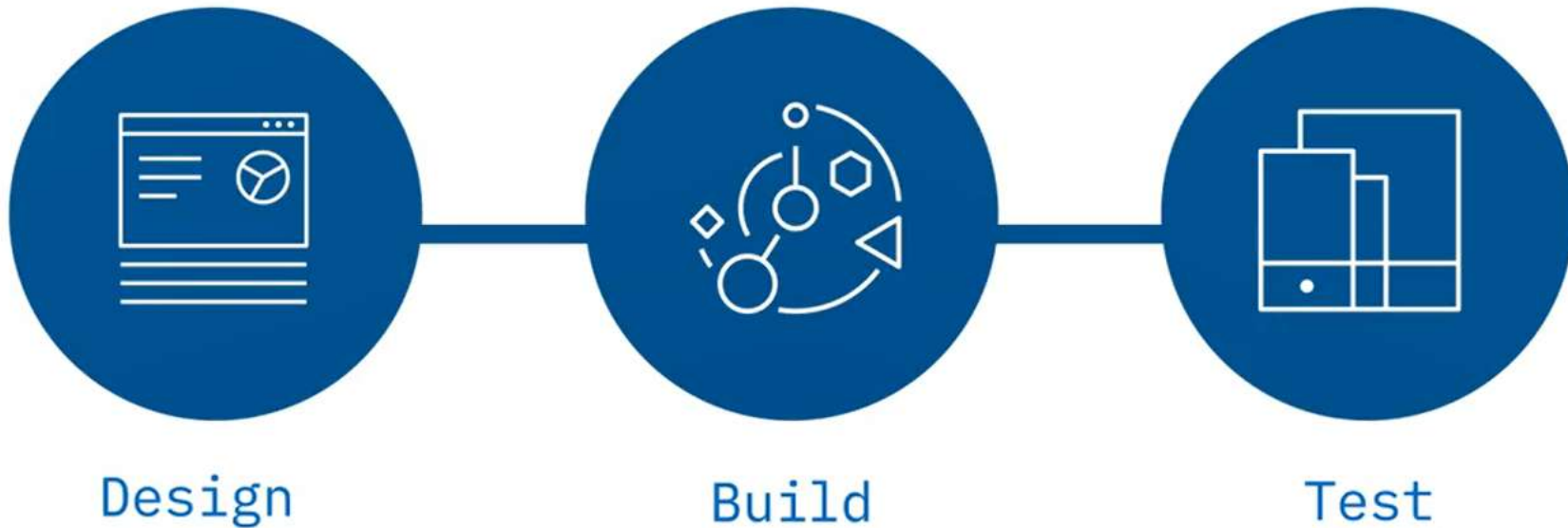
Module 5 – Job Opportunities and Skillsets in Software Engineering

Software engineering defined



Application of scientific principles

Software engineering defined



Systematic approach to software development

Software crisis



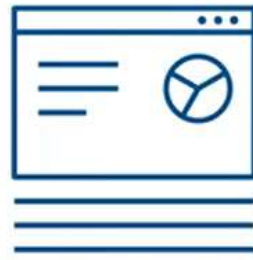
- Began in mid-1960s
- Resulted from software development that ran over budget and behind schedule with unmanageable and buggy code

Software crisis

Old Software Solutions



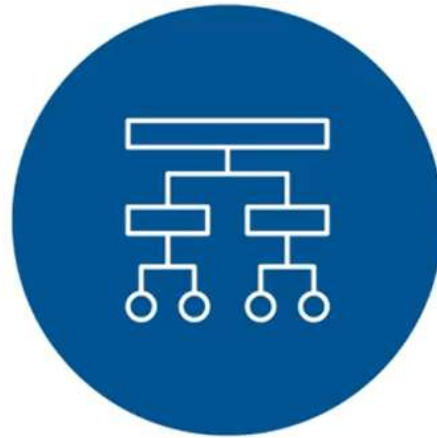
Newer Technologies



Solutions didn't scale to large projects

Software crisis

Engineering principles



Resolved as standardized methods in software engineering and computer-aided software engineering (CASE) tools were developed

Software crisis

1980s

CASE

Computer-aided software engineering



Business
analysis



Debugging
environments



Validation
tools



Configuration
management



Metrics &
measurement



Project
management

Software engineer vs software developer



Engineer

- Also developers
- Broad, big picture knowledge base
- Systematic development process
- Focus on structure



Developer

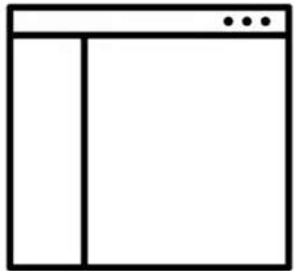
- Narrower in scope than SE
- Creative approaches
- Write code to implement specific functionality and solve specific problems

Software engineer responsibilities



- Design, build, and maintain software systems
- Write and test code
- Consult with stakeholders, third party vendors, security specialists, and other team members

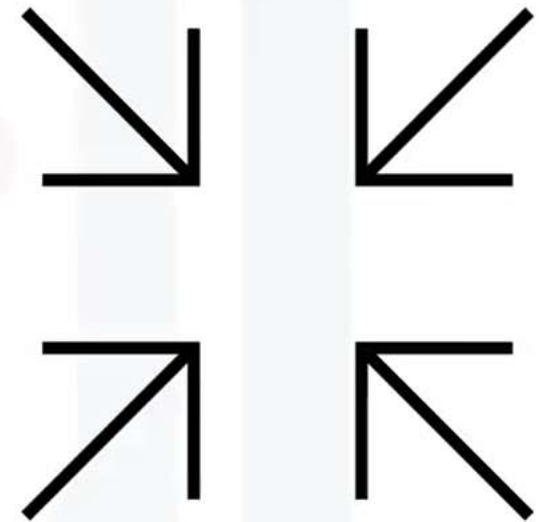
Software Development Life Cycle (SDLC)



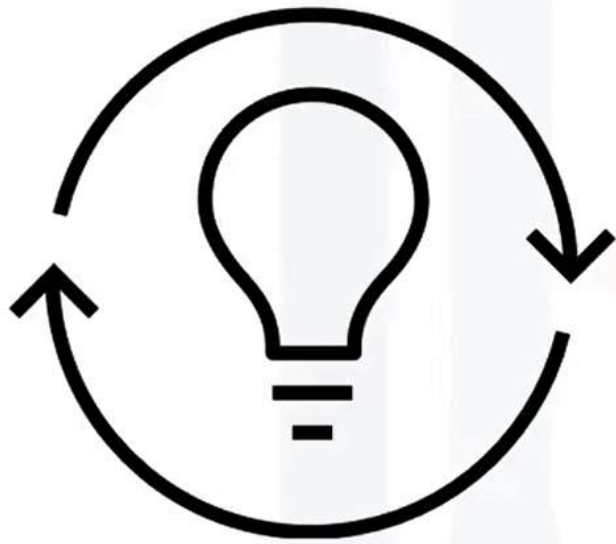
- Scientific approach to software development
- Guides the software development process
- Identifies discrete steps needed to develop software

Software development life cycle (SDLC)

- Systematic process to develop high-quality software
- Aims to produce software that meets requirements
- Defined phases with their own processes and deliverables
- Cycle of planning, design, and development
- Minimizes development risks and costs



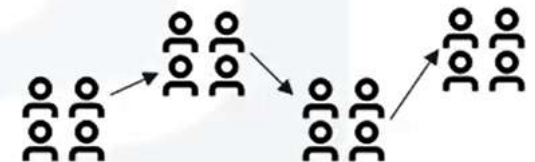
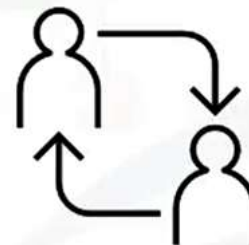
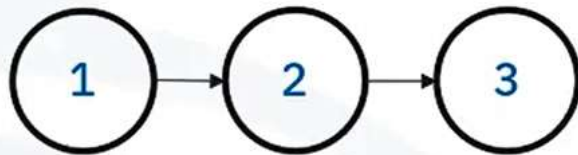
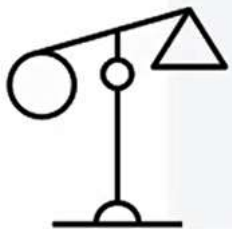
History of the SDLC



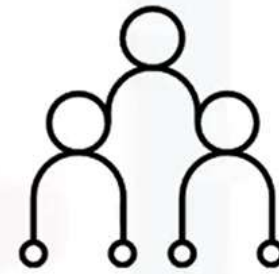
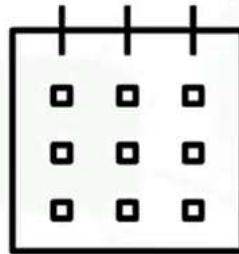
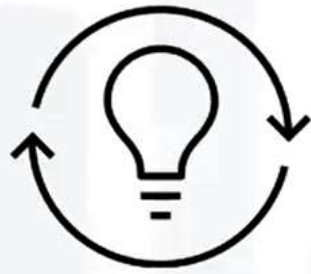
- Conceived of in the mid-1960s
- A deliberate approach needed to manage complex projects
- Initially used the waterfall method
- Adapted to use iterative methods

Advantages of the SDLC (1 of 2)

- Improves efficiency and reduce risks
- Team members know what they should be working on and when
- Facilitates communication among stakeholders
- Team members know when development can move to the next phase



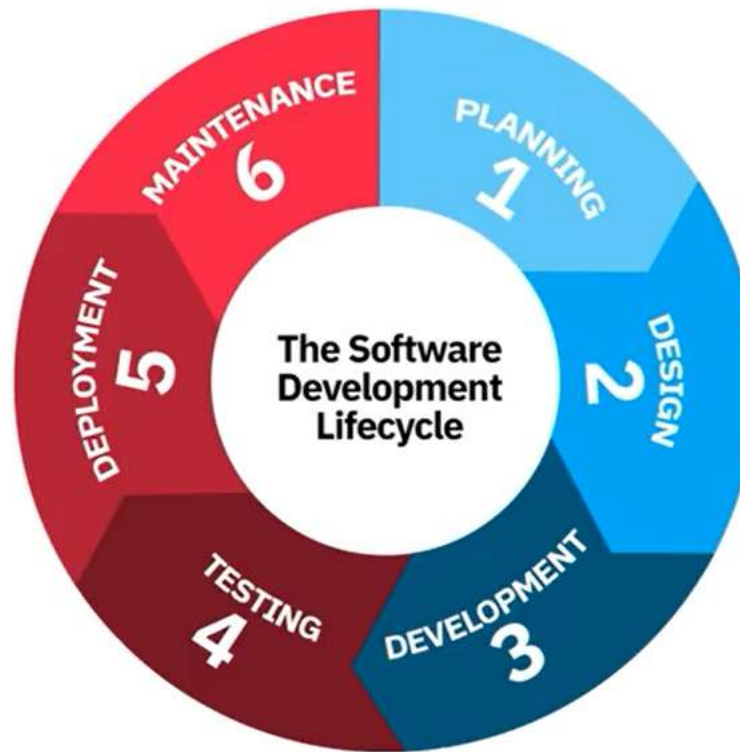
Advantages of the SDLC (2 of 2)



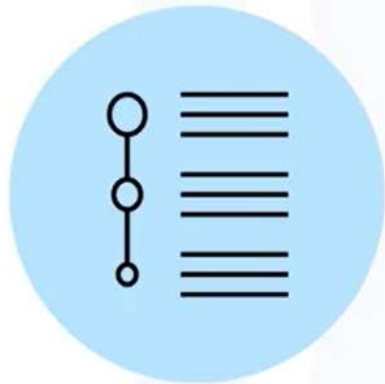
- Respond to changing requirements
- Solve problems early in the process
- Reduces overlapping responsibilities

Phases of the SDLC

- Organizations may have different names for each stage
- Some organizations have more or fewer stages

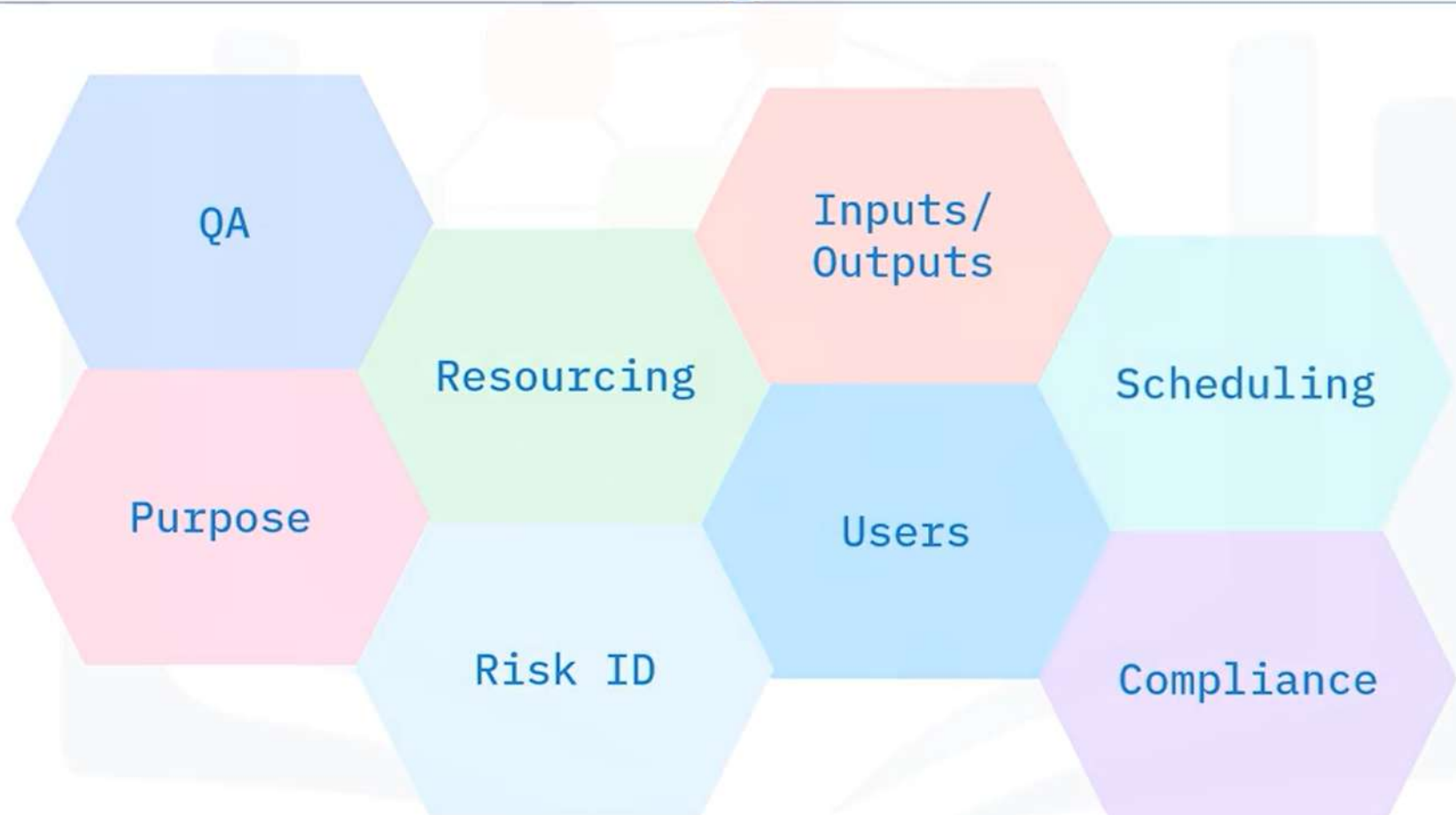


Phase 1: Planning



- Gathered
- Analyzed
- Documented
- Prioritized

Phase 1: Planning

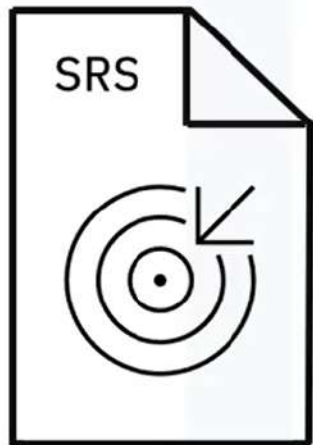


Prototyping



- Small-scale replica to clarify requirements
- Tests design ideas
- Can be developed at various stages of the SDLC

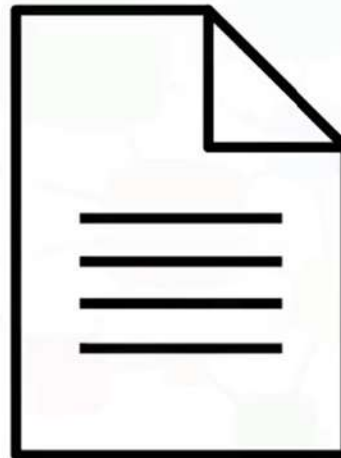
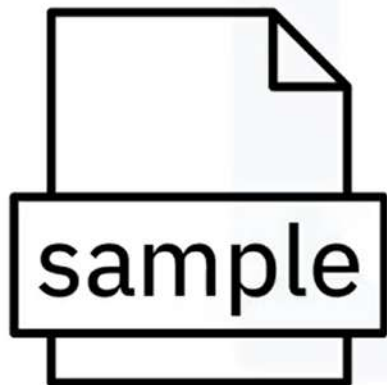
Software requirements specification



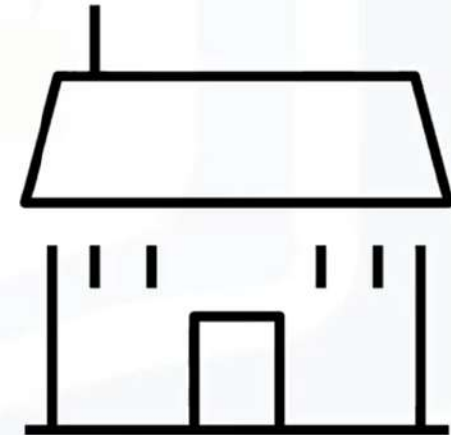
- Requirements are documented in the SRS
- All stakeholders must agree

Phase 2: Design

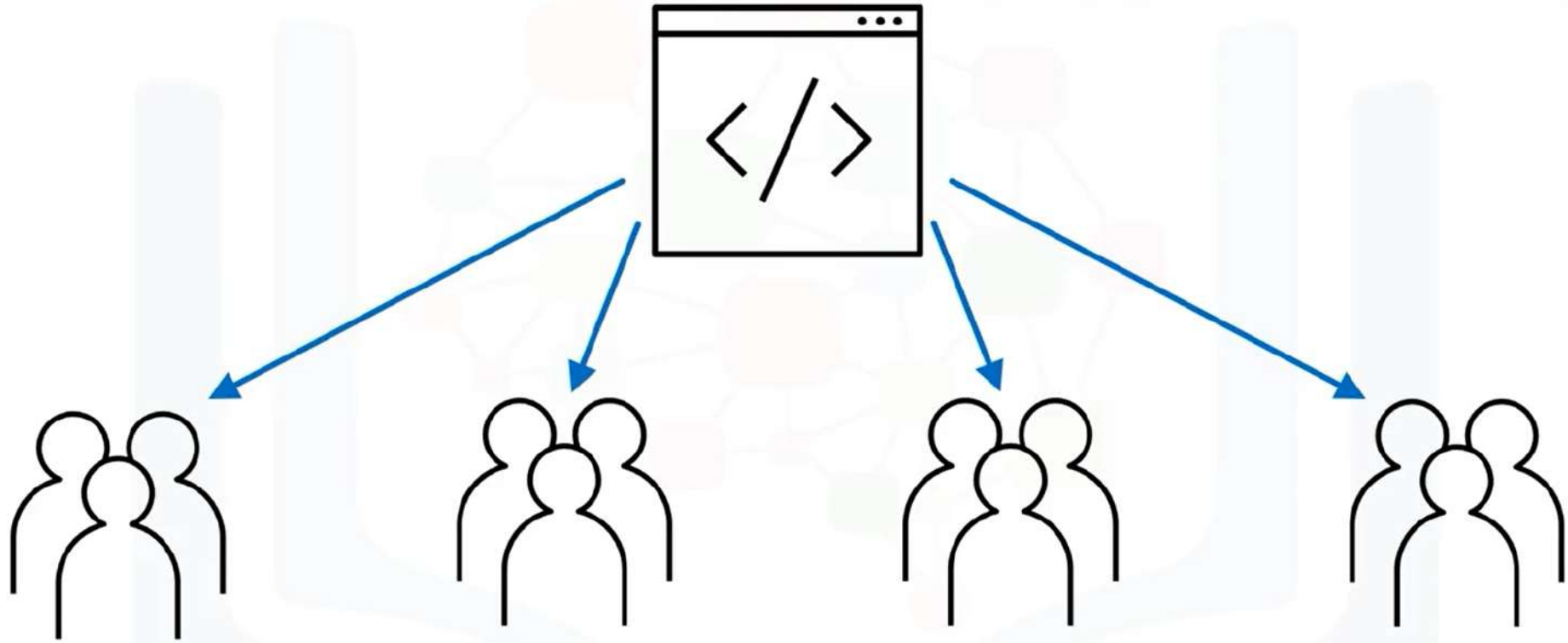
Design
Document



Requirements are
gathered from the SRS
to develop
architecture.



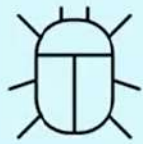
Phase 3: Development



Phase 4: Testing



Code is tested to ensure stability, security, and that it meets requirements from the SRS



Bugs reported, tracked, fixed, and retested

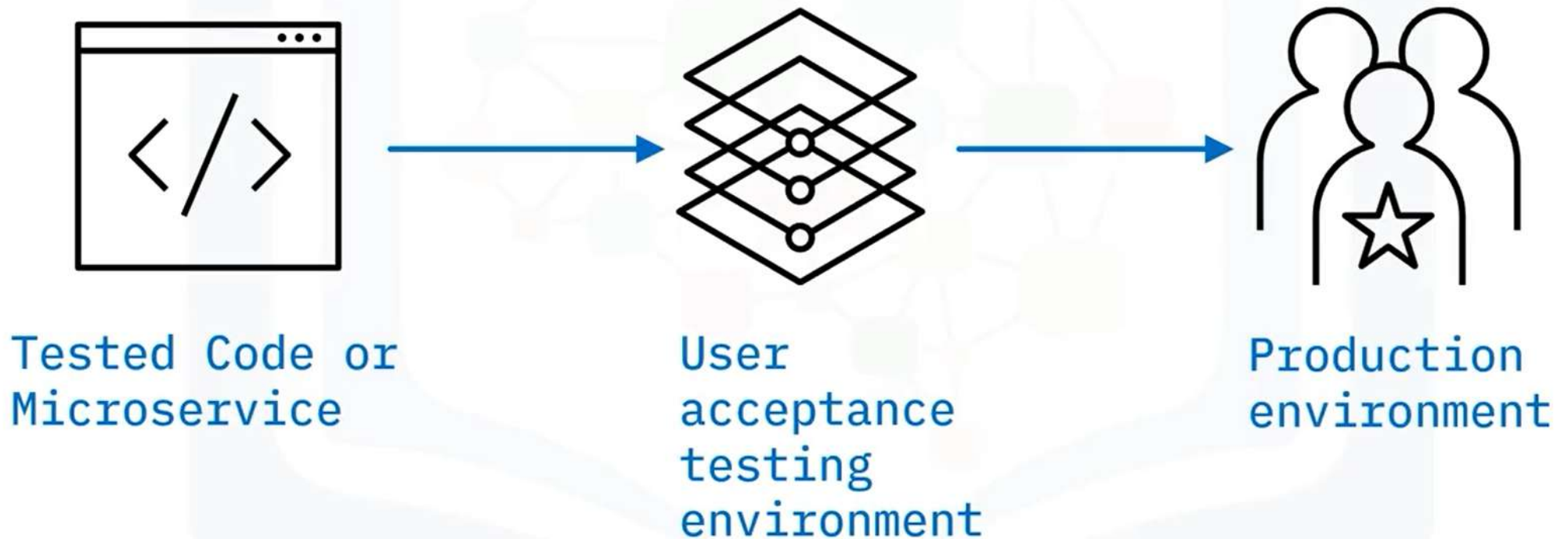
Unit

Integration

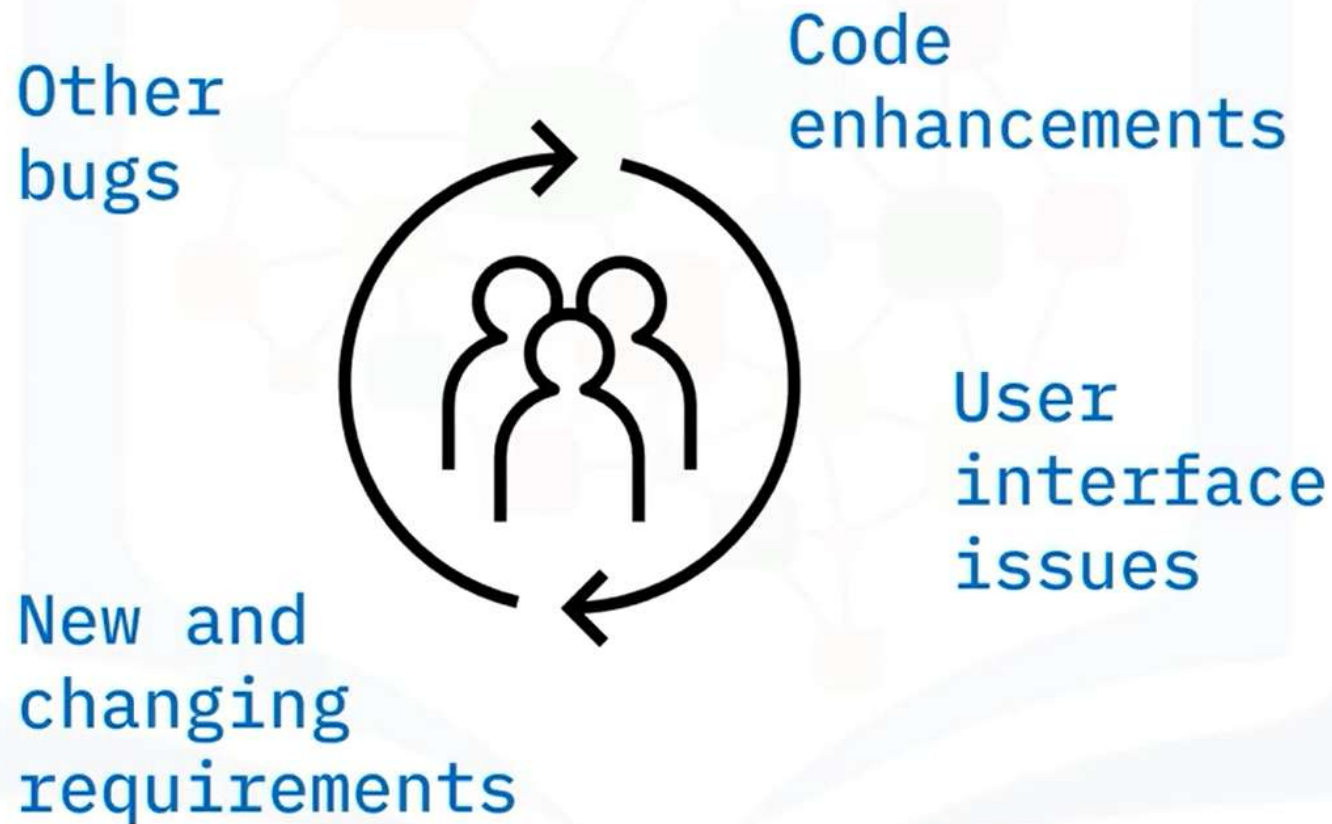
System

Acceptance

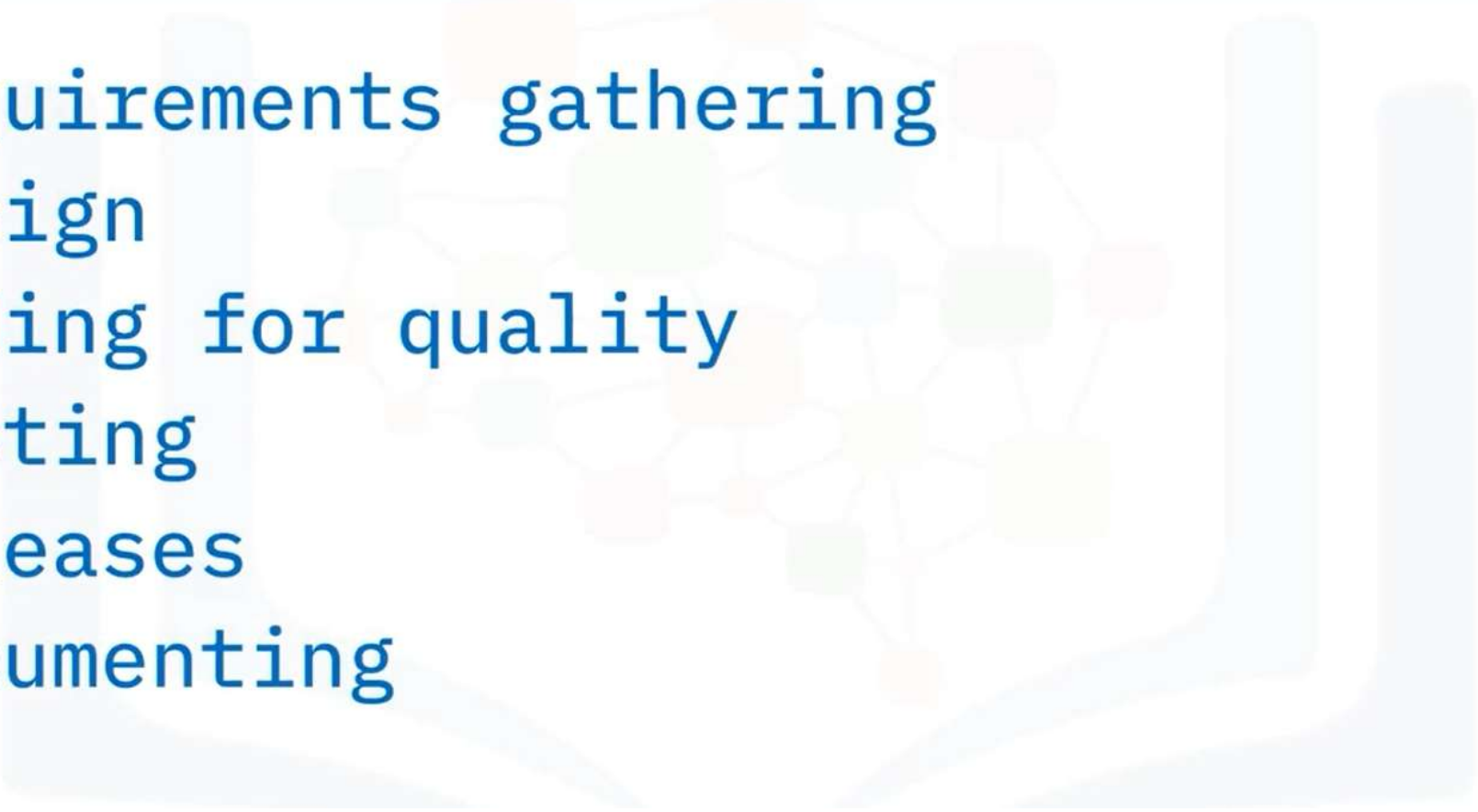
Phase 5: Deployment



Phase 6: Maintenance

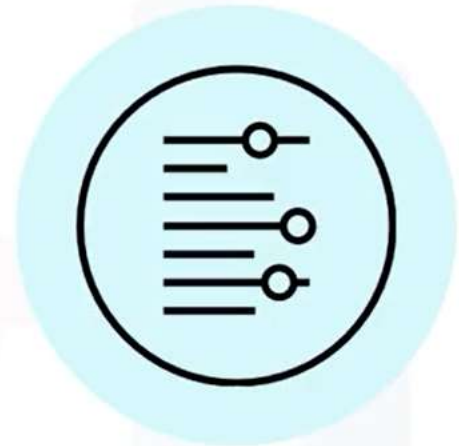


Common software engineering processes

- Requirements gathering
 - Design
 - Coding for quality
 - Testing
 - Releases
 - Documenting
- 

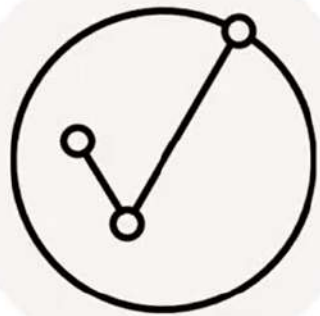
Requirement gathering

- The software requirements specification (SRS) encompasses the process of collecting and documenting the set of requirements that the software needs to adhere to
- It may include a set of use cases that describe the business needs and user flows that the software must implement

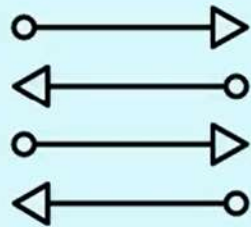


Requirement gathering

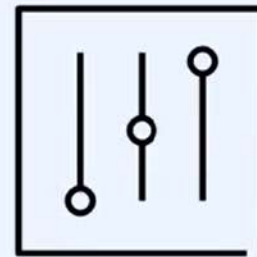
Software requirements can be classified into four broad categories:



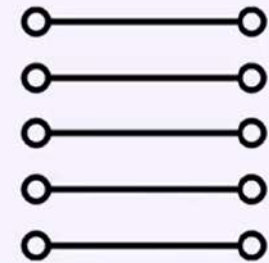
Functional



External &
user interface



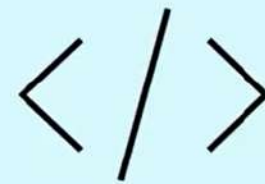
System
features



Nonfunctional

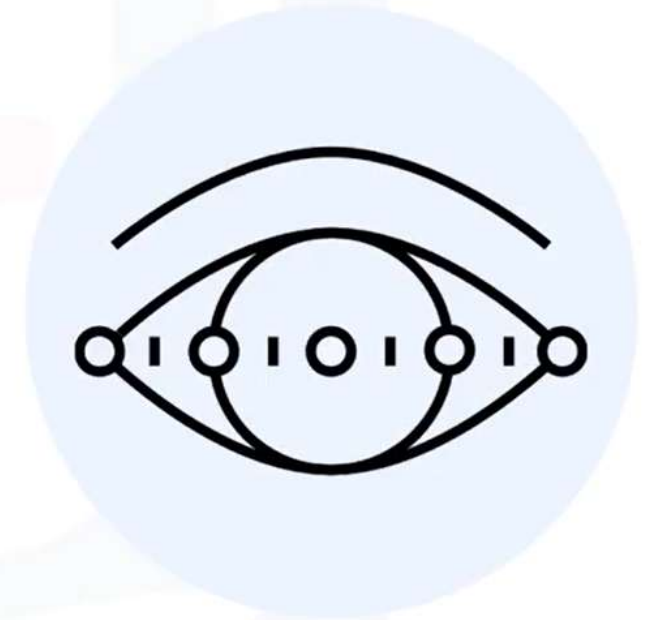
Design

- Transforming requirements into code
- Breaking down requirements into sets of related components
- Communicating business rules and application logic



Coding for quality

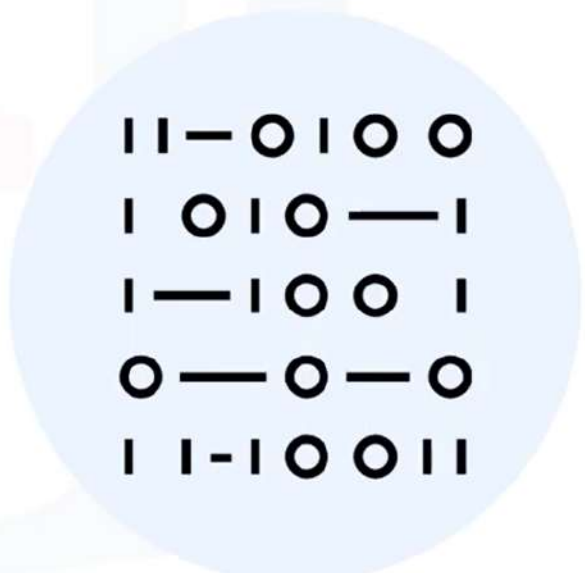
- ✓ Maintainability
- ✓ Readability
- ✓ Testability
- ✓ Security



Coding for quality

Quality code must fulfill the intended requirements of the software without defects

- Clean and consistent
- Easy to read and maintain
- Well documented
- Efficient



```
||-o|o o  
| o|o-|  
|-|o o |  
o-o-o  
| |-|o o||
```

Coding for quality

Coding for quality entails following a set of coding practices during development

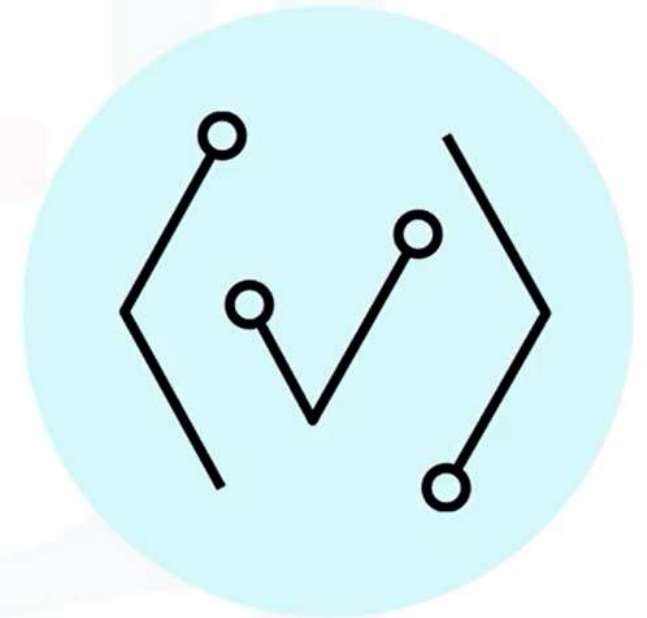
- Following coding standards
- Using linters to detect errors
- Commenting in the code itself to make it easy to understand and modify



Testing

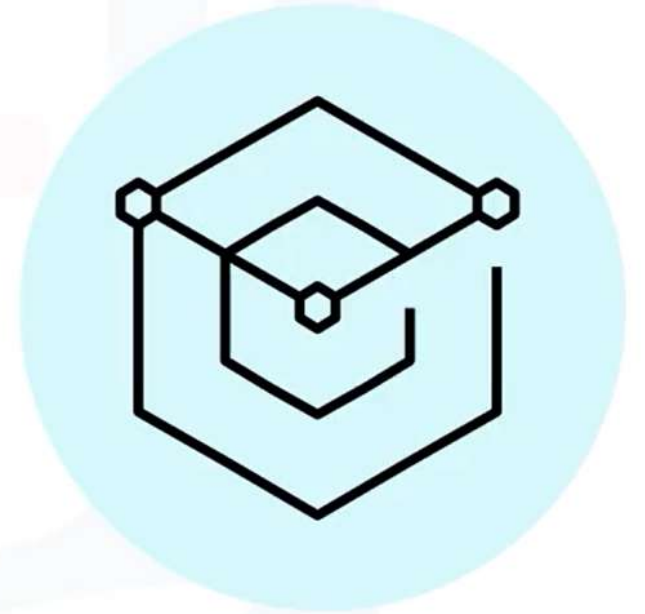
The process of verifying that the software matches established requirements and is free of bugs

- Identify errors, gaps, or missing requirements
- Ensures reliability, security, performance, and efficiency
- Software testing can often be automated or done manually



Testing

- Unit testing
- Integration testing
- System testing
- User acceptance testing (UAT) or Beta testing



Releases

Alpha

- Select stakeholders
- May contain errors
- Preview of functioning version
- Design changes may occur

α

Beta

- All stakeholders
- User testing
- Meets requirements

β

General Availability

- Stable
- All users

GA

Documenting

- **System documentation**

README files, inline comments, architecture and design documents, verification information and maintenance guides

- **User documentation**

User guides, instructional videos, manuals, online and inline help

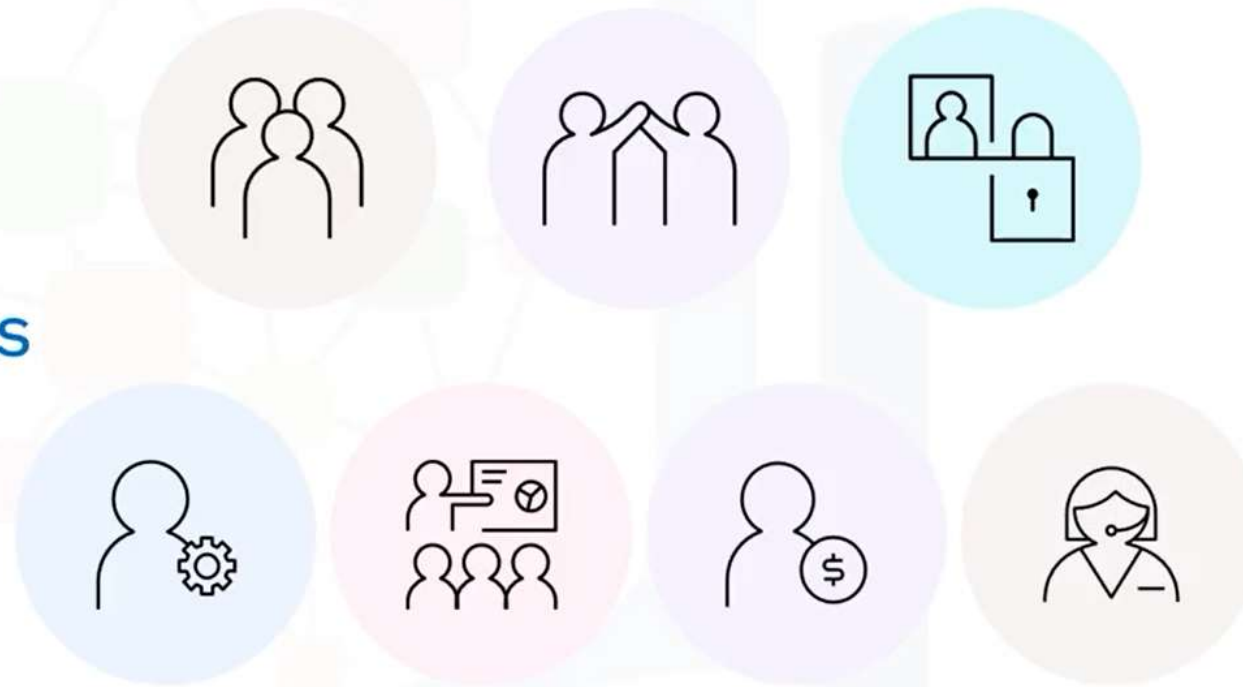
Steps to gathering requirements

- Identifying stakeholders
- Establishing goals and objectives
- Eliciting requirements from the stakeholders
- Documenting the requirements
- Analyzing and confirming the requirements
- Prioritizing

Identifying stakeholders

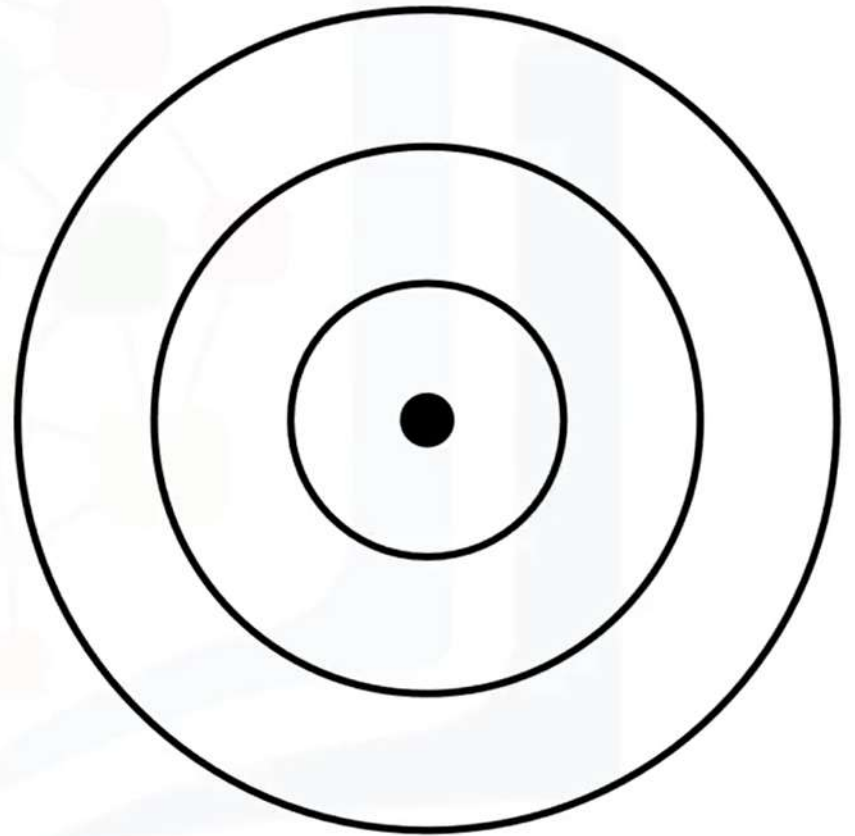
Key personnel:

- Decision-makers
- End-users
- System administrators
- Engineering
- Marketing
- Sales
- Customer support



Establishing goals and objectives

- **Goals:** broad, long-term achievable outcomes
- **Objectives:** actionable, measurable actions that achieve the goal



Eliciting, documenting, confirming

- **Elicit**

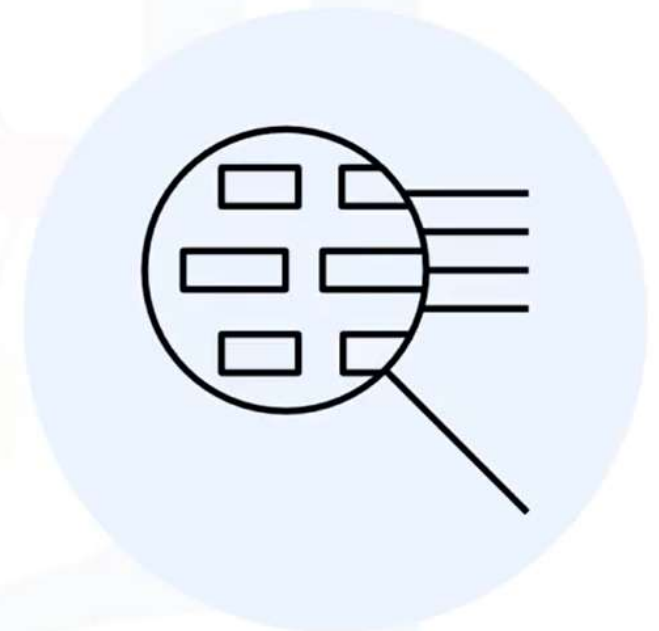
- Surveys
- Questionnaires
- Interviews

- **Document**

- Align with goals and objectives
- Easily understood

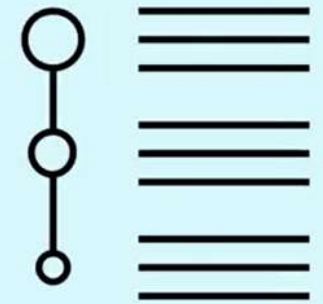
- **Confirm**

- Consistency
- Clarity
- Completeness



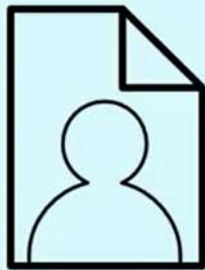
Prioritizing

- Must-have
- Highly desired
- Nice to have



Requirements documentation

- Software requirements specification (SRS)
- User requirements specification (URS)
- System requirements specification (SysRS)



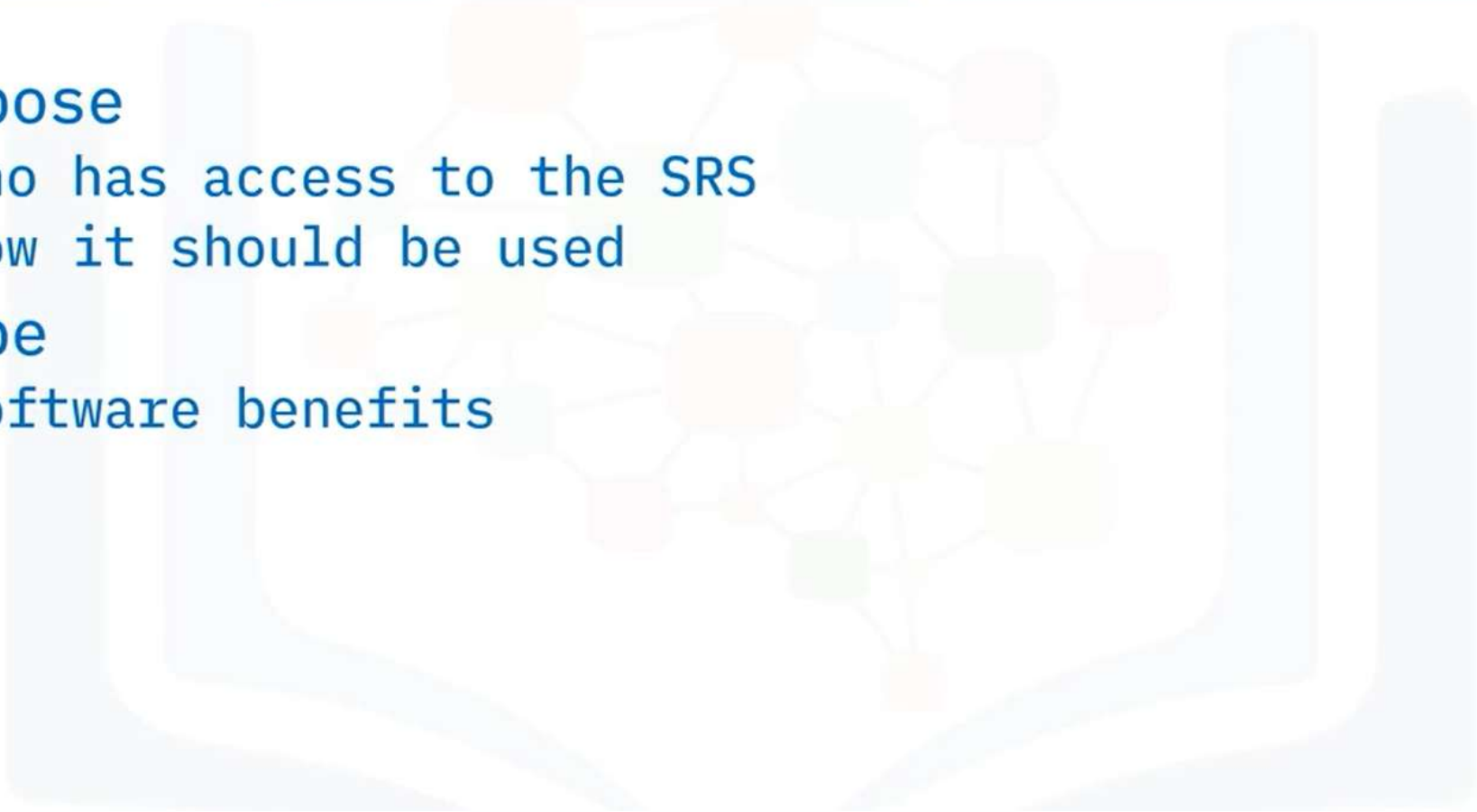
Software requirements specification (SRS)

- Captures functionalities the software should perform
- Establishes benchmarks / service-levels for performance
- Purpose and scope
- Constraints, assumptions, dependencies
- Requirements
 - Functional
 - External interface
 - System features
 - Non-functional



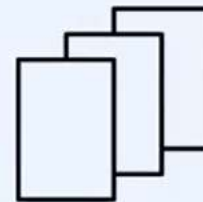
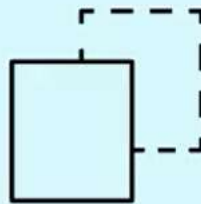
SRS: Purpose and scope

- Purpose
 - Who has access to the SRS
 - How it should be used
- Scope
 - Software benefits



SRS: Constraints, assumptions, dependencies

- **Constraints:** how the software must operate under given conditions
- **Assumptions:** required OS or hardware
- **Dependencies:** on other software products

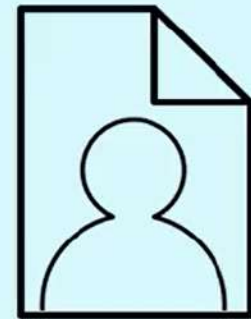


SRS: Requirements

- **Functional:** functions of the software
- **External:** users and interactions with other hardware or software
- **System features:** functions of the system
- **Non-functional:** performance, safety, security, quality

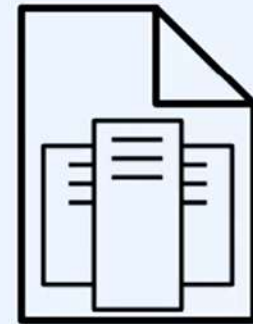
User requirements specification (URS)

- Describe business need and end-user expectations
- **User stories:**
 - Who is the user?
 - What is the function that needs to be performed?
 - Why does the user want this functionality?
- Confirmed during user acceptance testing
- Often combined into the SRS



System Requirement Specification (SysRS)

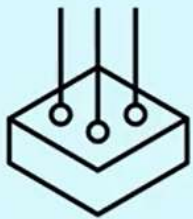
- Outlines requirements of the system
- Broader than an SRS
- Contains:
 - System capabilities
 - Interfaces and user characteristics
 - Policy
 - Regulation
 - Personnel
 - Performance
 - Security
 - System acceptance criteria
 - Hardware expectations



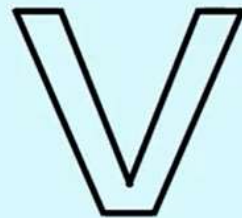
Common development methodologies

A process is needed to clarify communication and facilitate information sharing among team members.

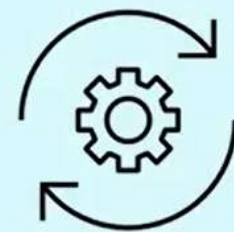
Waterfall



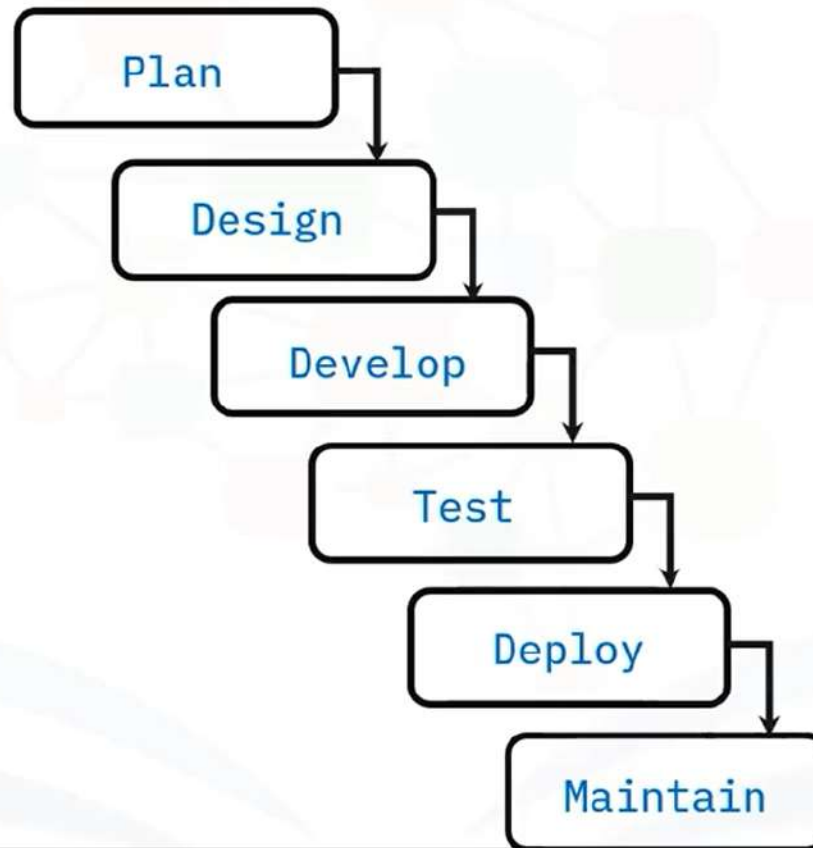
V-shape model



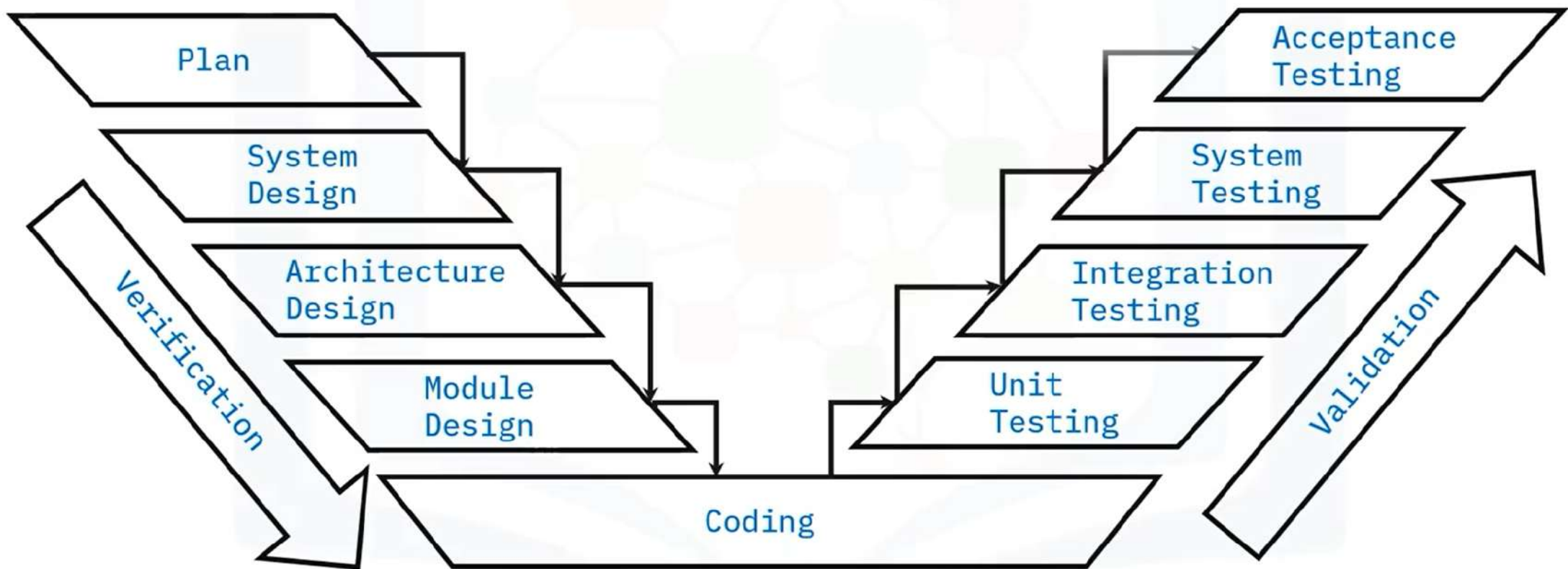
Agile



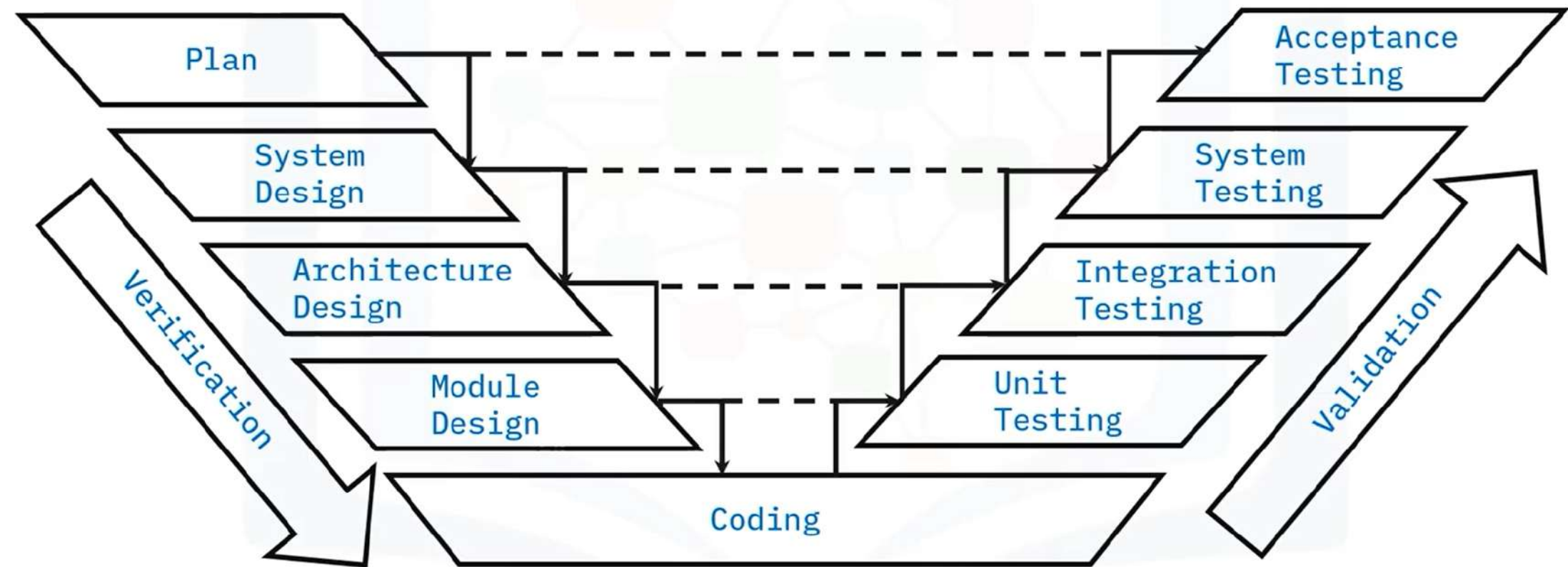
Waterfall method



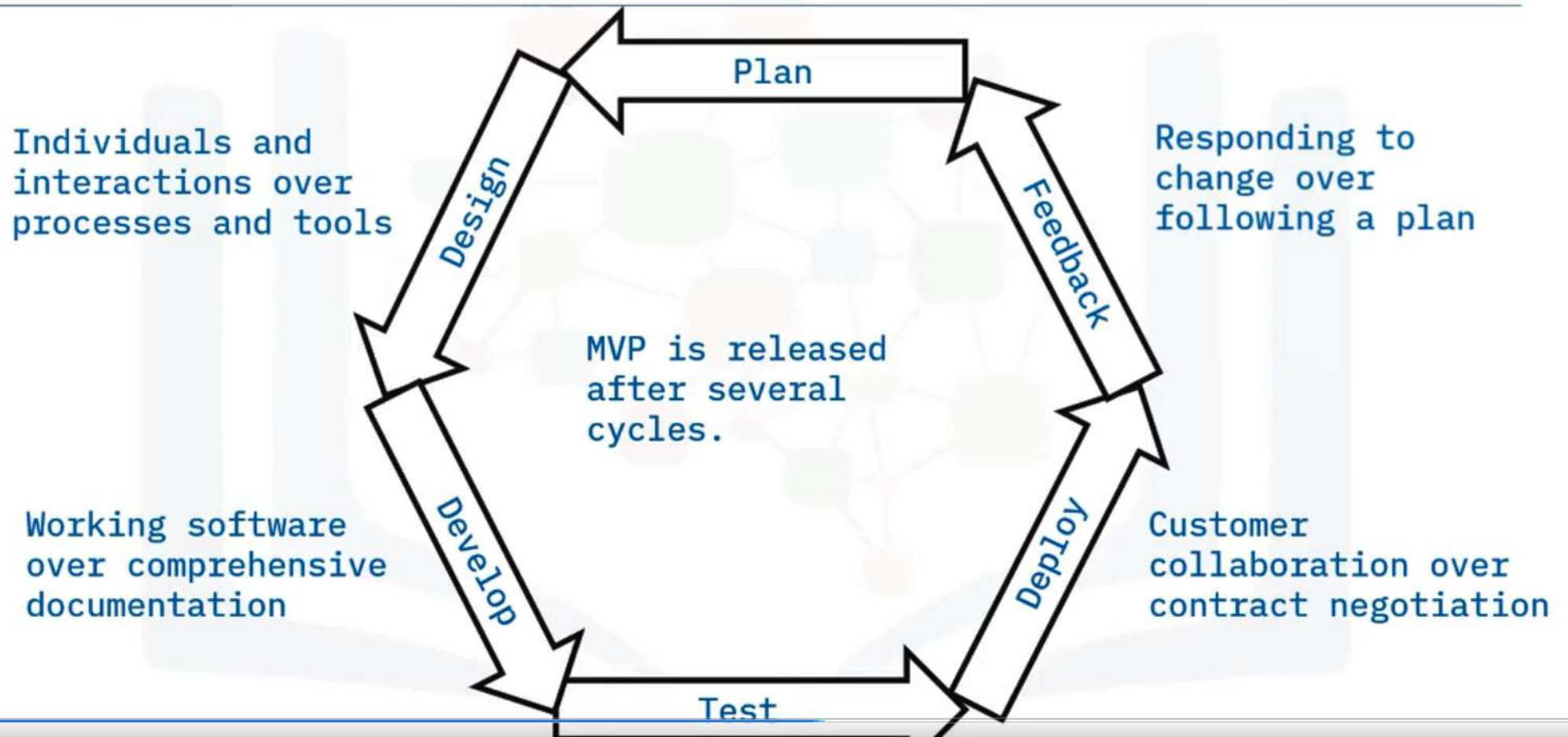
V-shape model



V-shape model



Agile



Sequential vs iterative

Sequential: Waterfall and V-Shape Model



Waterfall pros and cons



Pros

- Team members understand their responsibilities due to discrete, well-defined stages
- Easier to estimate budget and allocate resources



Cons

- Lacks flexibility
- Change is hard to accommodate

V-shape model pros and cons



Pros

- Easy to use
- Test plans designed upfront saves development and testing time



Cons

- Rigid
- Does not accommodate changing requirements

Agile pros and cons



Pros

- Changing requirements handled easily
- Feedback incorporated regularly

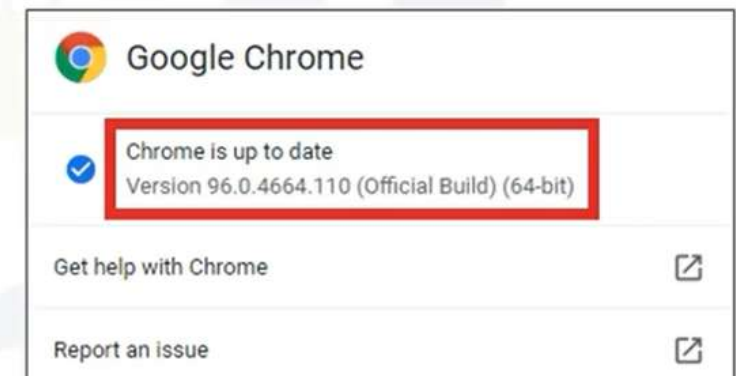
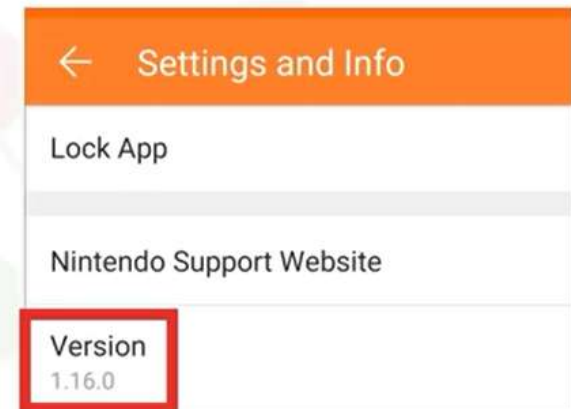


Cons

- Budgeting and resource allocation is challenging
- Project scope not clearly defined

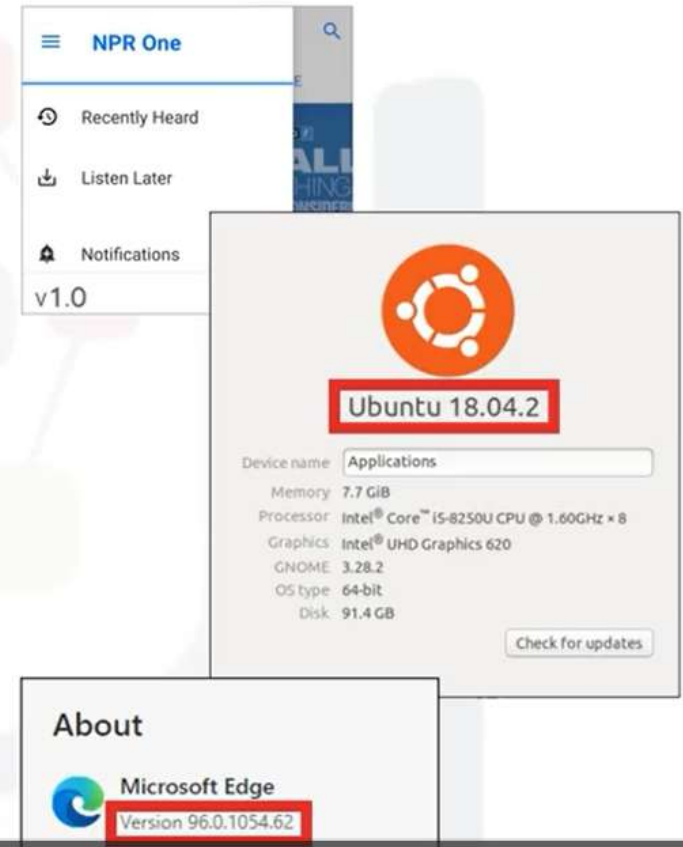
Software versions

- Software versions are identified by version numbers
- Version numbers indicate:
 - When the software was released
 - When it was updated
 - If any minor changes or fixes were made to the software
- Software developers use versioning to keep track of new software, updates, and patches



Version numbers

- Version numbers can be short or long, with 2, 3, or 4 sets
- Each number set is divided by a period
- An application with a 1.0 version number indicates the first release
- Software with many releases and updates will have a larger number
- Some use dates for versioning, such as Ubuntu Linux version 18.04.2 released in 2018 April, with a change shown in the third number set



What do version numbers mean?

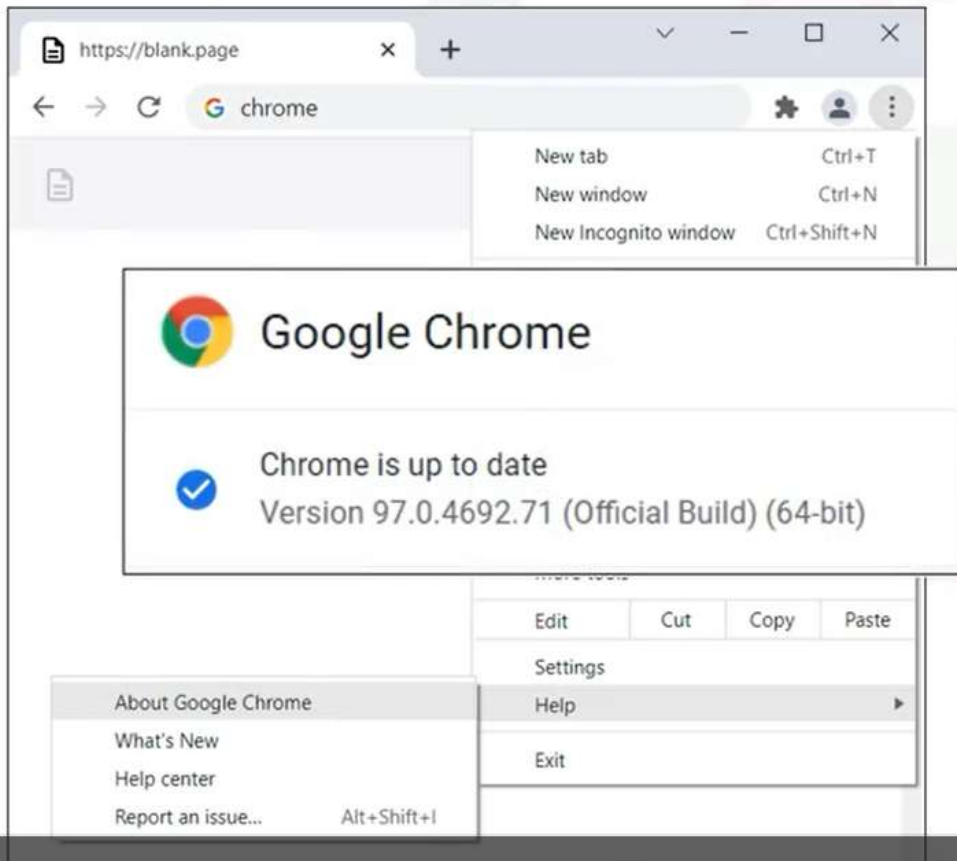
Some version numbers follow the semantic numbering system and have 4 parts separated by a period

- The first number indicates major changes to the software, such as a new release
- The second number indicates that minor changes were made to a piece of software
- The third number in the version number indicates patches or minor bug fixes
- The fourth number indicates build numbers, build dates, and less significant changes

About

version 9.1.33.6

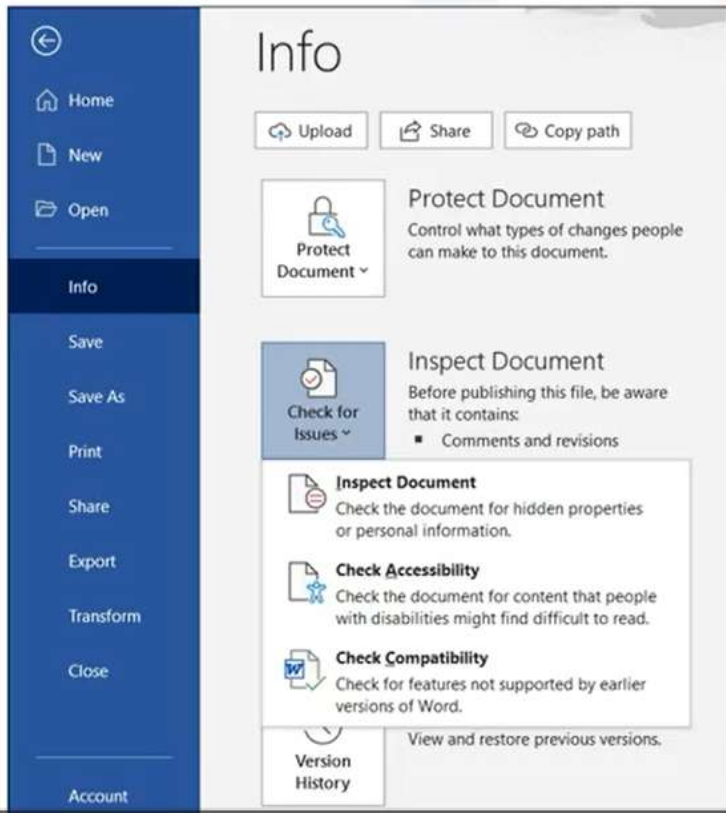
Software version numbers



Practice viewing your software version numbers in a web browser:

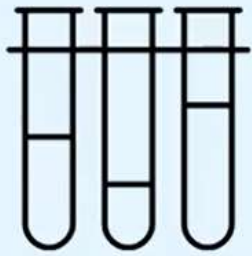
- Select the three dots or three lines in the top-right corner of your browser
- Select the menu item Help
- Select About to view the version information
- The version of your web browser will display

Version compatibility



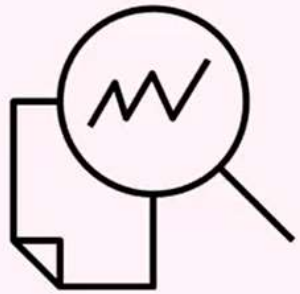
- Older versions may not work as well in newer versions
- Compatibility with old and new versions of software is a common problem
- Troubleshoot compatibility issues by viewing the software version
- Update software to a newer version that is compatible
- Backwards-compatible software functions properly with older versions of files, programs, and systems

What is software testing?



- Integrate quality checks throughout SDLC
- Purpose
 - Ensure software meets requirements
 - Error-free software

Test cases



Verify
functionality
and
requirements

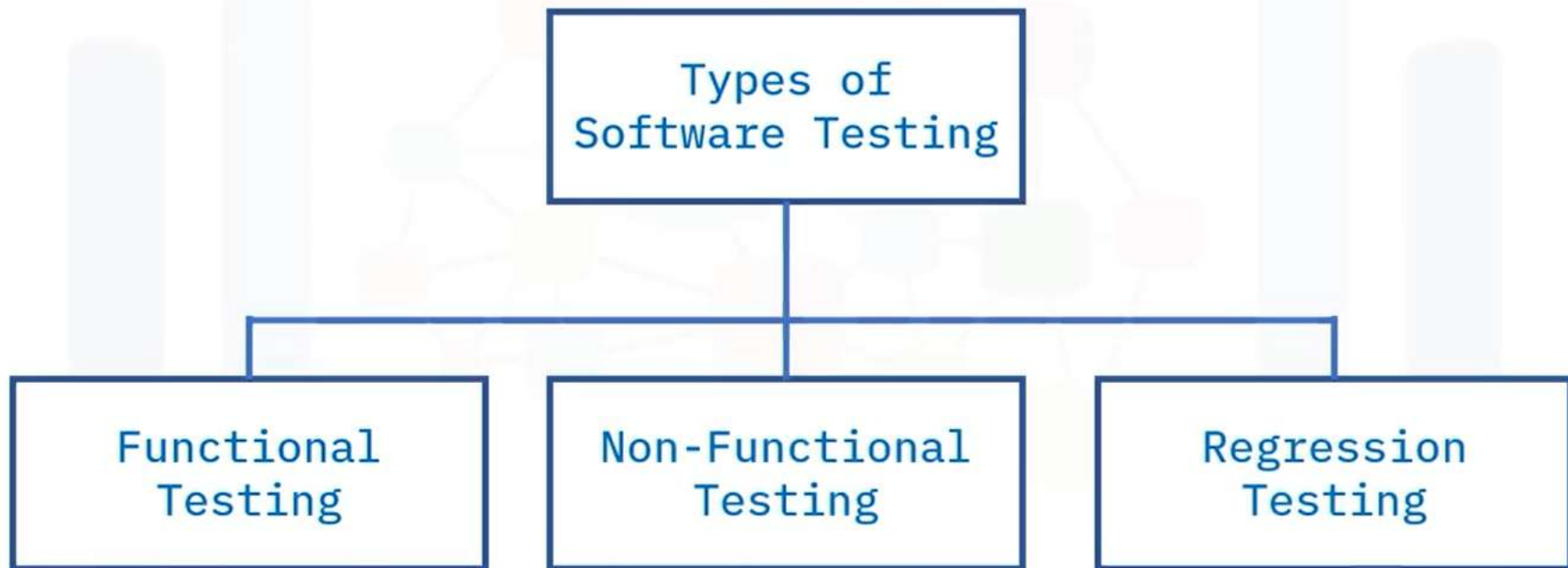
Steps

Data

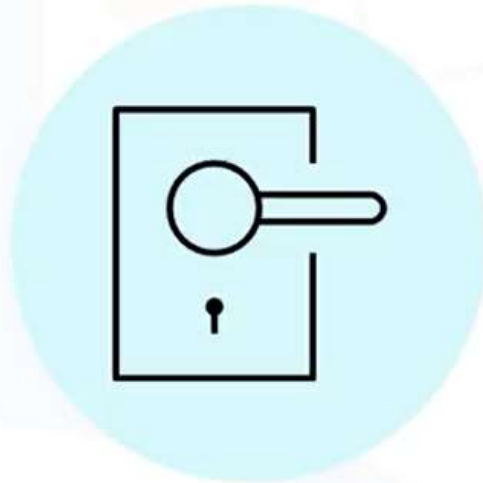
Inputs

Expected
Output

Three types of testing



Functional testing

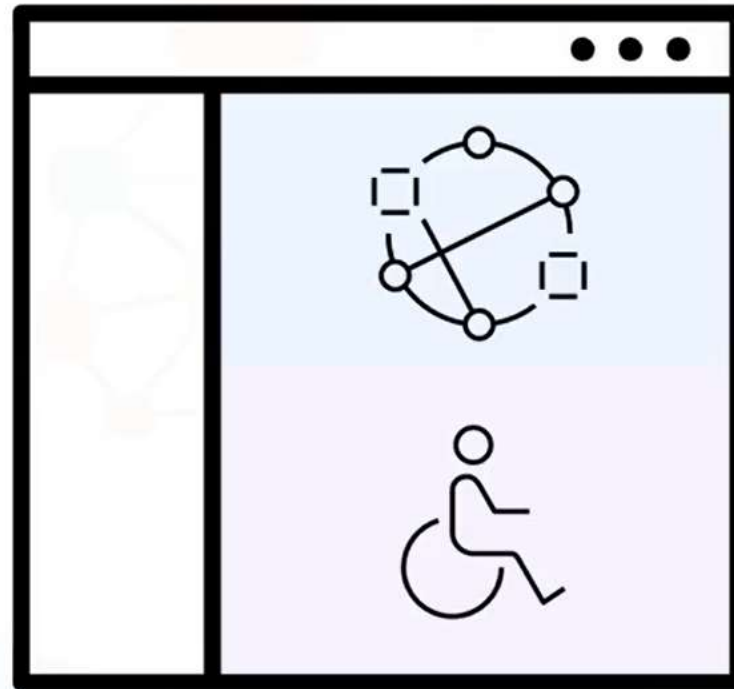
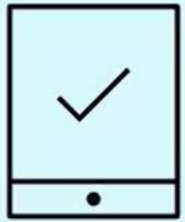


Manually



Automated

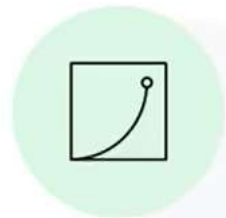
Purpose of functional testing



Usability

Accessibility

Non-functional testing attributes



Performance



Security



Scalability



Availability

Non-functional testing questions



- How does the application behave under stress?
- What happens when many users log in at the same time?
- Are instructions consistent with behavior?

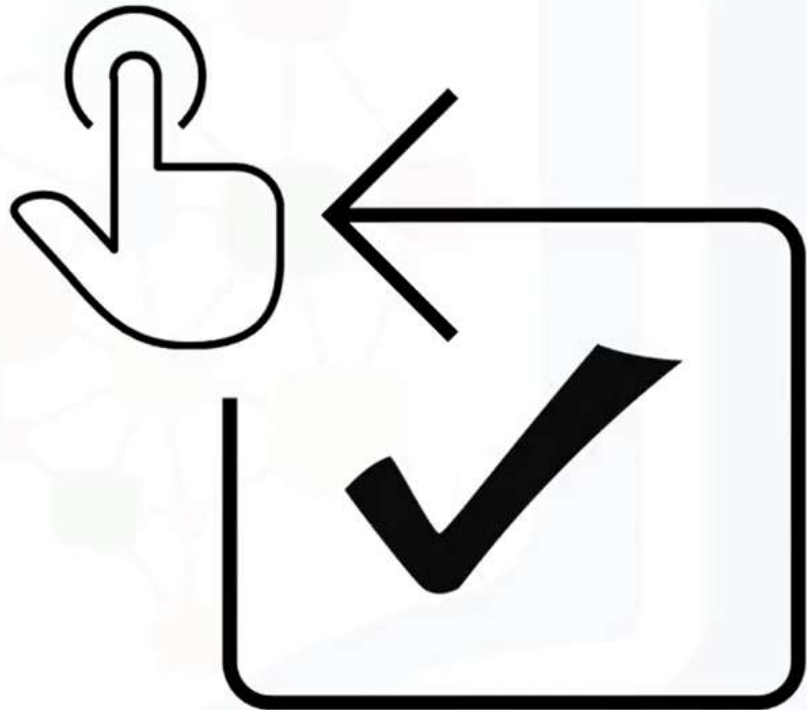
Non-functional testing questions



- How does the application behave under different OSs?
- How does the application handle disaster recovery?
- How secure is the application?

Regression testing

- Confirms changes don't break the application
- Occurs after fixes such as a change in requirements or when defects are fixed

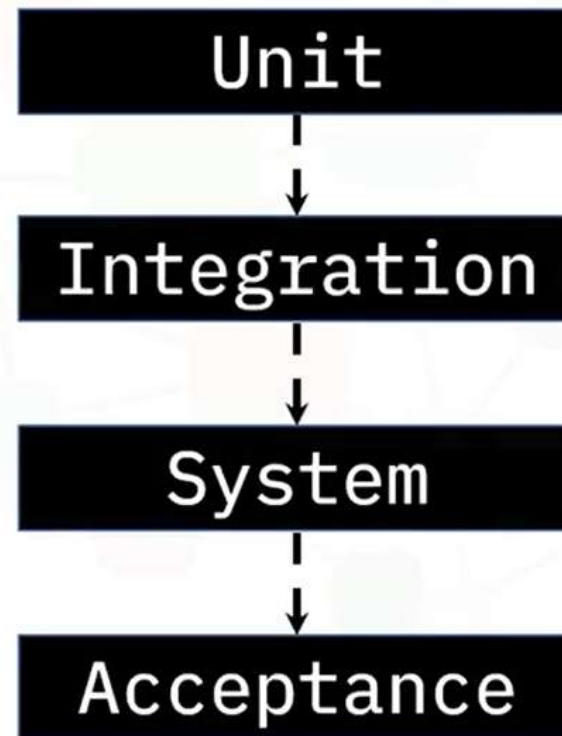


Choosing test cases for regression testing

Choose test cases that contain:



Testing levels



Unit testing

- Test a module of code
- Occurs during the build phase of the SDLC
- Eliminate errors before integration with other modules

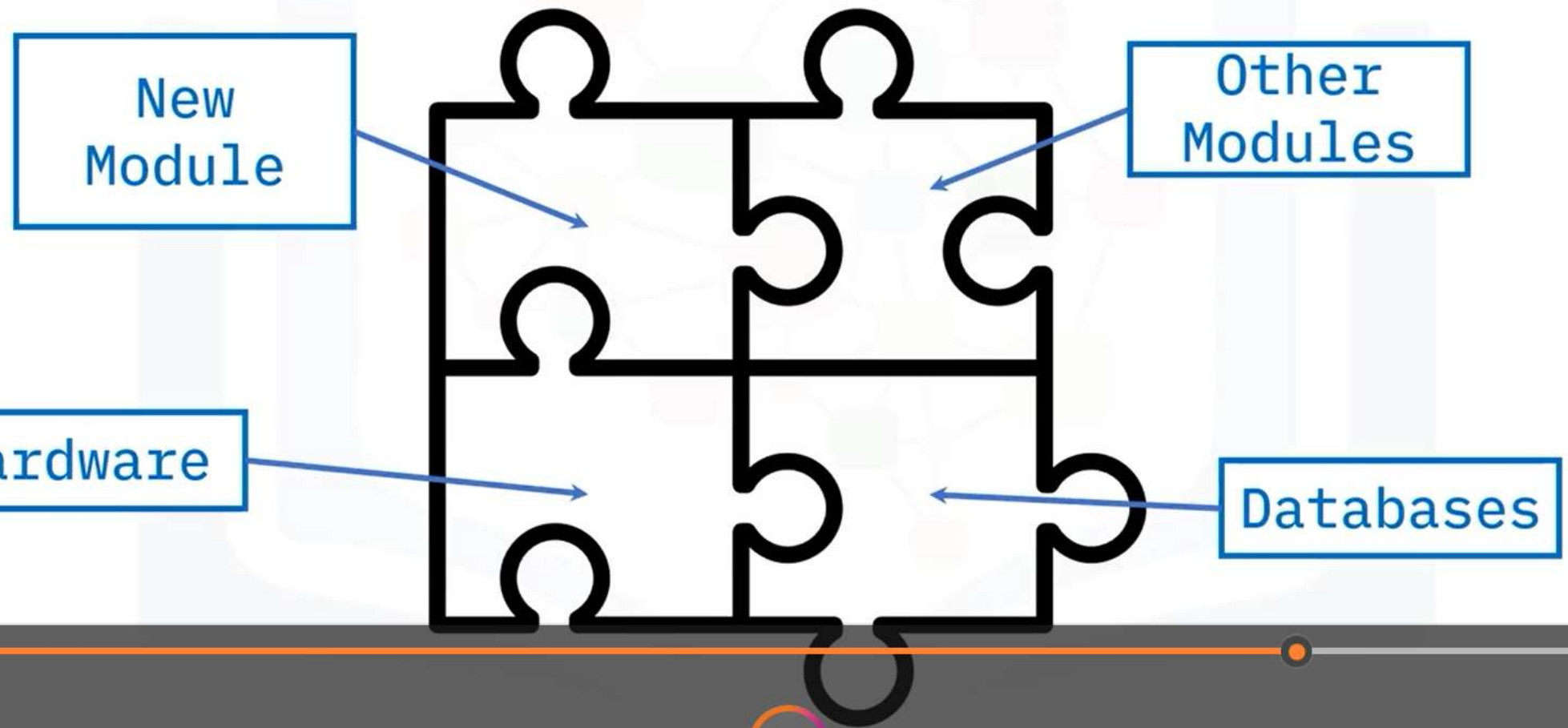


Integration testing



- Identify errors introduced when two or more modules are combined
- Type of black-box test
- Occurs after modules are combined into the larger application

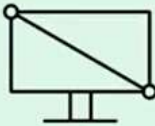
Purpose of integration testing



System testing



Compliance with SRS



Functional and
Non-functional



Validate the system



Staging environment

Acceptance testing



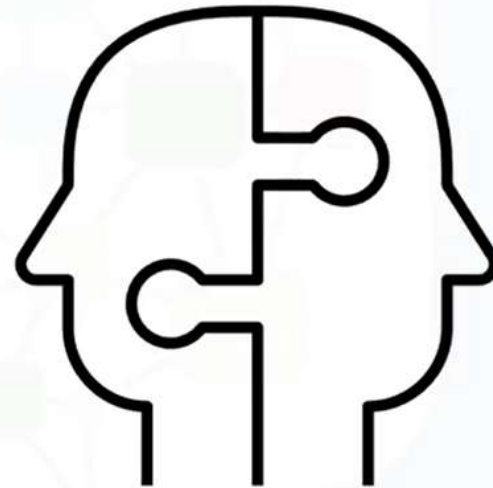
Users



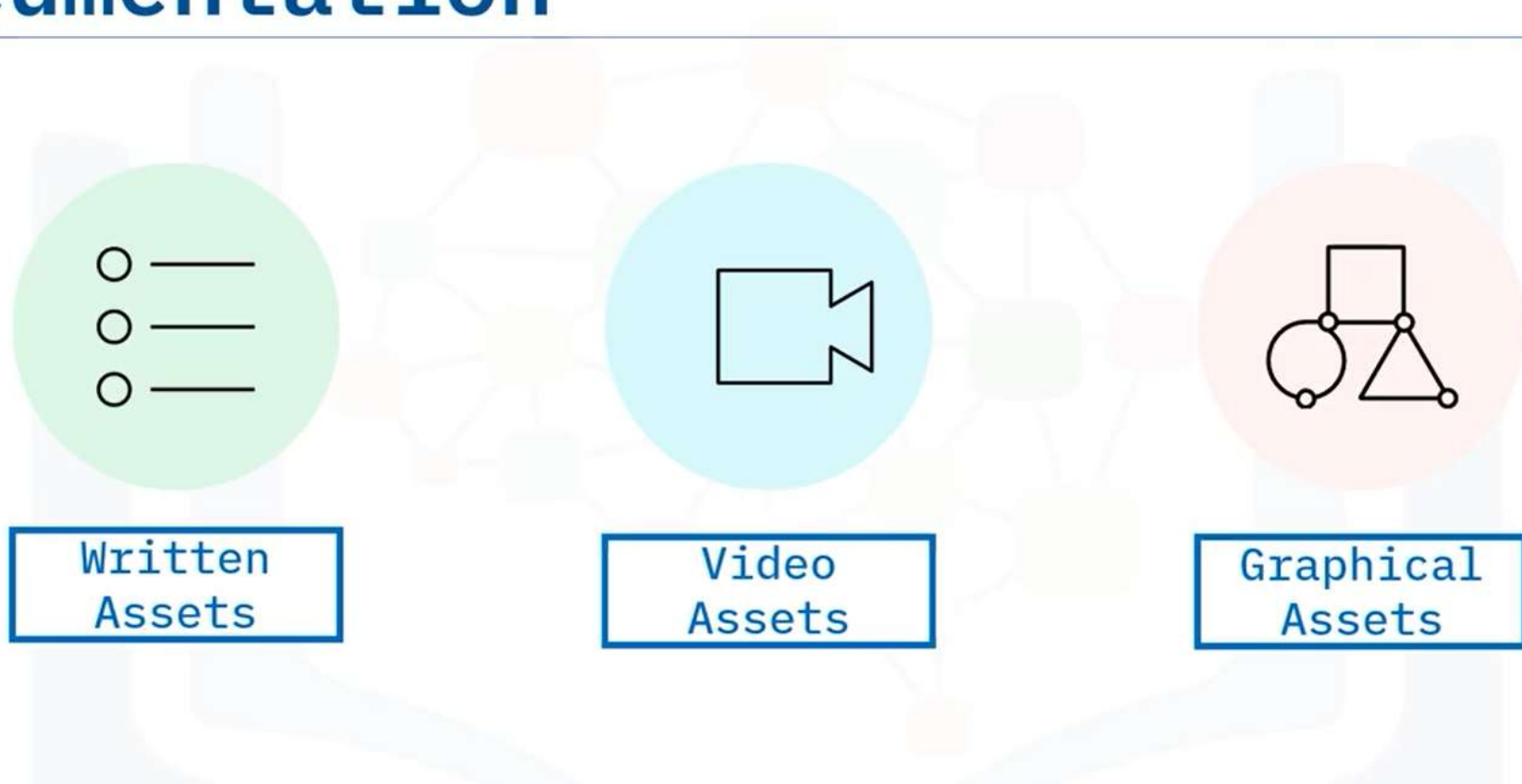
Customers



Stakeholders

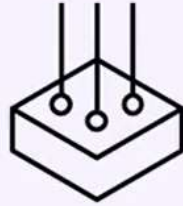


Documentation



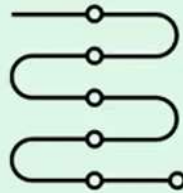
Product vs process documentation

Product
Documentation



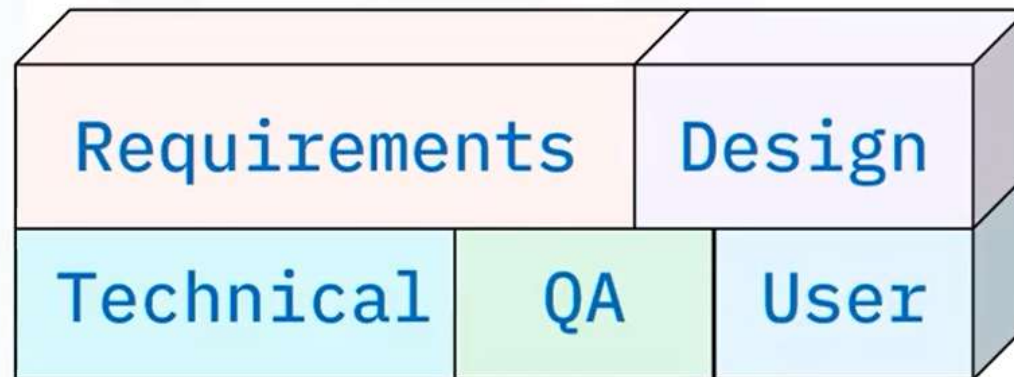
Relates to product functionality

Process
Documentation



Describes how to complete a task

Types of product documentation



Requirements documentation

Intended for the development team including developers, architects, and QA. Describes expected features and functionality

Software requirements specifications, system requirements specification, and user acceptance specifications

Design documentation

Written by architects and development team to explain how the software will be built to meet requirements

Consists of both conceptual and technical design documents

Technical documentation

Written in the code to help developers read the code

Comments embedded in code and working papers that explain how the code works, documents that record ideas and thoughts during implementation

Quality assurance documentation

Pertains to the testing team's strategy
progress, and metrics

Test plans, test data, test scenarios,
test cases, test strategies, and
traceability matrices

User documentation

Intended for end-users to explain how to operate software or help install and troubleshoot system

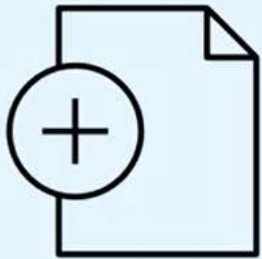
FAQs, installation and help guides, tutorials, and user manuals

Standard operating procedures

- Accompanies process documentation
- Step-by-step instructions on how to accomplish common yet complex tasks
- Ex: organization specific instructions for check in code to a repository
- Types of SOPs:
 - Flowcharts
 - Hierarchical
 - Step-by-step



Updating documentation



- Must be kept up to date
- Documentation should be reviewed and updated periodically



Software development project roles

- 
1. Project manager / Scrum master
 2. Stakeholder
 3. System / Software architect
 4. UX Designer
 5. Developer
 6. Tester / QA engineer
 7. Site reliability / Ops engineer
 8. Product manager / Product owner
 9. Technical writer / Information developer

Project manager / Scrum master

Project manager

- Planning, scheduling, and budgeting
- Allocating personnel and resources
- Executing the software plan
- Team communication

Scrum master

- Ensure team success
- Individual success
- Prioritize people over process
- Focused on facilitating communication between all team members

Stakeholder

Interested parties who are affected by the software product

- Customer, end-users, decision-maker

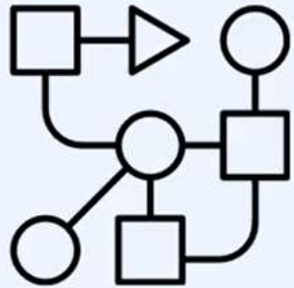


Responsibilities:

- Defines requirements
- Provides feedback on requirements implementation
- May participate in beta and acceptance testing

System architect

Designs, describes, and communicates architecture of a project to team members

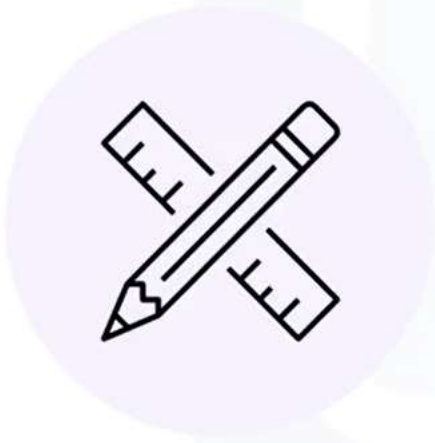


Responsibilities:

- Designs inner structure
- Designs technical aspects
- Provides technical support regarding the architecture

UX designer

Balances making software interface intuitive yet also robust



Responsibilities:

- How software communicates functionality to the user
- How the user interacts with the software

Developer

Writes the code that powers the software

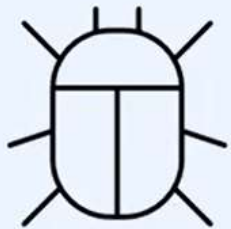


Responsibilities:

- Implement architecture in design documents
- Incorporate SRS requirements
- Employ UX design

Tester / QA engineer

Ensures quality of the product by testing to see if the software meets requirements



Responsibilities:

- Writes and execute test cases
- Provides feedback to development teams

Site reliability engineer

Bridges software development and operations

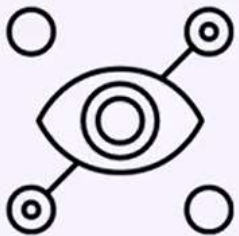


Responsibilities:

- Tracks and communicates incidents
- Automates systems, procedures, and processes
- Trouble shoot
- Ensures product reliability

Product manager / Product owner

Understands requirements and end-user needs



Responsibilities:

- Leads development efforts
- Ensures product provides value to the customer

Technical writer / Information developer

Writes documentation for end-user

Technical material → Non-technical audience



Responsibilities:

- Writes user manuals
- Writes reports
- Writes white papers
- Writes press releases